
The impact of transactions costs and slippage on algorithmic trading performance

September 30, 2024

ROMAIN LORAS

Abstract

In algorithmic trading, transaction costs and slippage are critical factors that can significantly affect strategy performance. This study examines how these hidden expenses reduce profitability by eroding returns. Transaction costs include fees, commissions, and bid-ask spreads, while slippage occurs when an order is executed at a less favorable price than expected due to market dynamics. Both factors are particularly influential in high-frequency strategies where small price movements are targeted. By incorporating realistic models of transaction costs and slippage into backtesting frameworks, traders can better understand the true performance of their strategies and develop robust, cost-aware algorithms. The analysis highlights the importance of optimizing execution strategies to minimize these impacts, ultimately improving the consistency and sustainability of trading outcomes.

Contents

1	Introduction	4
2	Transaction costs and slippage	6
2.1	The importance of costs and slippage	6
2.2	Transaction Costs	6
2.3	Slippage	8
2.4	Impact on Trade Execution and Returns	9
3	Impact on algorithmic trading performance	11
3.1	High-Frequency Strategies	11
3.2	Low-Frequency Strategies	12
3.3	Impact on Backtest Accuracy	12
3.4	Comparative Sensitivity	13
4	Mitigation Techniques	15
4.1	Execution Strategies	15
4.2	Buffering	16
4.2.1	Mathematical Formulation of Buffering	16
4.2.2	Impact on Transaction Costs	17
4.3	Trade Optimizations	17
4.4	Strategy Example	18
4.5	Moving Average Strategy with Buffering	18
5	Comments on the Backtest Results	25
6	Conclusion	27

Disclaimer

The content of this document is for informational and educational purposes only and should not be construed as financial advice. The strategies and methodologies discussed herein are based on historical data and may not accurately predict future performance. Past performance is not indicative of future results, and no representation is made that any strategy will achieve profits or avoid losses.

The information contained in this document may include inaccuracies or typographical errors. While efforts have been made to ensure the reliability and accuracy of the data, no guarantees can be provided. Users are encouraged to conduct their own research and due diligence before making any financial decisions or implementing any strategies discussed.

Backtesting results are derived from historical data and should be viewed with caution. The performance of a trading strategy during a backtest may not reflect real-world trading conditions, including market volatility, slippage, and transaction costs. Real-time trading may result in different outcomes than those suggested by backtested results due to unforeseen factors, including changes in market dynamics and economic conditions.

The author and associated parties shall not be held liable for any losses or damages arising from the use of this document or reliance on the information contained herein. The content is intended for individuals who are knowledgeable about the risks of trading and are aware of the potential for both gains and losses.

By reading this document, you acknowledge and accept the risks associated with trading and the inherent uncertainties in financial markets. It is recommended that you consult with a qualified financial advisor or professional before making any investment decisions.

This document is not an offer or solicitation to buy or sell any securities or financial instruments. All trading activities involve substantial risks, and you should only trade with capital that you can afford to lose.

1 Introduction

Algorithmic trading has become a cornerstone of modern financial markets, leveraging advanced computational techniques to automate the buying and selling of assets based on predefined strategies. By removing human emotion from trading decisions and relying on data-driven approaches, algorithmic trading allows for the rapid execution of orders and the ability to capitalize on even the smallest market inefficiencies. The widespread adoption of algorithmic strategies by institutional investors and hedge funds has led to the evolution of more sophisticated models, including high-frequency trading (HFT), arbitrage, and trend-following systems.

However, while algorithmic trading offers significant advantages, it is not without challenges. One of the most critical considerations that can erode profitability is the impact of transaction costs and slippage. Transaction costs, which include brokerage fees, commissions, and bid-ask spreads, can accumulate quickly, particularly for strategies that involve frequent trades. The costs may seem negligible on a per-trade basis, but they compound over time and can severely reduce overall returns.

Slippage, another key factor, refers to the difference between the expected price of a trade and the actual price at which it is executed. This occurs when market conditions change between the time a trade is initiated and when it is filled. In fast-moving markets or with large order sizes, slippage can become a significant cost, especially for high-frequency traders who rely on precision and tight margins. The faster the market moves or the larger the order, the greater the risk of slippage, often turning a potentially profitable trade into a losing one.

These hidden costs are particularly problematic for high-frequency trading (HFT) strategies, which target small price discrepancies. Even

slight variations in execution prices or underestimated transaction costs can disrupt the profitability of these strategies. As a result, understanding the cumulative effects of transaction costs and slippage is essential for traders seeking to build sustainable and effective trading systems.

Incorporating these factors into backtesting frameworks is mandatory. Many trading algorithms perform well in theoretical simulations, but when subjected to real-world market conditions, their profitability may diminish due to the overlooked impact of costs. Traders must ensure that backtests reflect realistic scenarios, including dynamic market liquidity, changing bid-ask spreads, and execution delays, to avoid overestimating potential returns.

Moreover, robust execution strategies can help minimize the effects of transaction costs and slippage. Techniques such as optimal order placement, smart order routing, and the use of limit orders instead of market orders can significantly reduce these hidden costs. By doing so, traders not only improve the accuracy of their backtesting but also enhance the overall efficiency and profitability of their trading systems.

In this study, we examine how transaction costs and slippage affect algorithmic trading performance and present methods for mitigating their impact. By developing cost-aware algorithms and incorporating these expenses into the backtesting process, traders can gain a more realistic understanding of their strategies' true potential. This analysis will provide insights into optimizing execution methods and developing trading systems that are better equipped to navigate the complexities of real market conditions.

2 Transaction costs and slippage

2.1 The importance of costs and slippage

In algorithmic trading, understanding the nature and impact of transaction costs and slippage is critical to developing profitable strategies. While many trading algorithms may appear profitable in theoretical backtests, failing to account for these hidden costs can lead to significant underperformance in live markets. This section defines transaction costs and slippage, their various sources, and how they can negatively impact trade execution and overall returns.

2.2 Transaction Costs

Transaction costs refer to the expenses associated with executing trades in the financial markets. These costs vary depending on factors such as the type of asset, trading venue, and order size. Common types of transaction costs include:

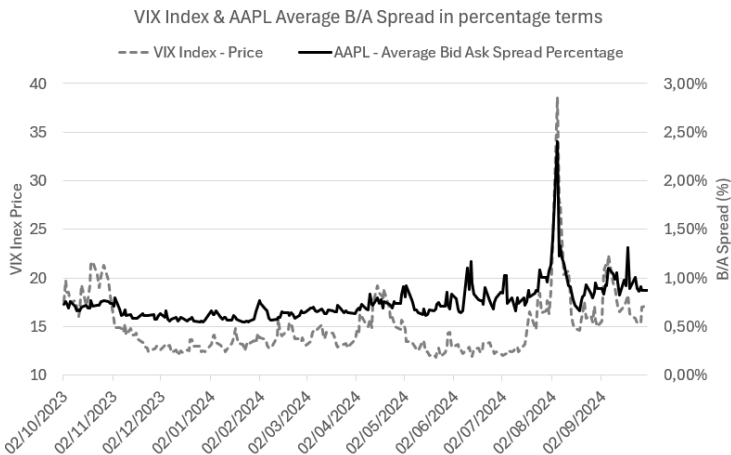
Brokerage Fees and Commissions:

Most brokers charge fees or commissions for executing trades on behalf of clients. In traditional markets, these fees may be a fixed rate per trade or a percentage of the trade value (we can also find tiered commission setup, an example could be found at IBKR). Although commission rates have dropped significantly with the rise of low-cost brokers and electronic trading platforms, they still accumulate over time, particularly for strategies that involve frequent trading.

Bid-Ask Spread:

The bid-ask spread is the difference between the price at which a buyer

is willing to purchase an asset (the bid) and the price at which a seller is willing to sell it (the ask). In liquid markets with high trading volumes, the spread tends to be narrow, reducing the cost of entering and exiting positions. However, in less liquid markets or during periods of high volatility, spreads can widen considerably, increasing the cost of trading. For high-frequency traders, these small spreads add up over time and can erode profits.



Market Impact:

Large orders can affect the price of an asset as they are executed, especially in markets with low liquidity. This market impact can cause the price to move unfavorably, leading to higher transaction costs. As traders attempt to buy or sell large quantities of an asset, they may drive the price up (in the case of buying) or down (in the case of selling), increasing the effective cost of the trade.

Clearing and Settlement Fees:

In some markets, additional fees are incurred when trades are processed and settled through clearinghouses. Although less significant than brokerage fees or bid-ask spreads, these costs can add up over time.

2.3 Slippage

Slippage occurs when a trade is executed at a price that differs from the trader's expected price. This often happens in fast-moving markets or when there is insufficient liquidity to fill an order at the desired price. Slippage can have a detrimental impact on performance, especially for strategies that rely on precise entry and exit points. There are several sources of slippage:

Market Volatility:

During periods of high volatility, prices can change rapidly between the time a trade is initiated and when it is executed. This price movement can lead to slippage, where the trade is filled at a less favorable price. Volatility can be particularly problematic for large orders or when trading illiquid assets, where price fluctuations are more pronounced.

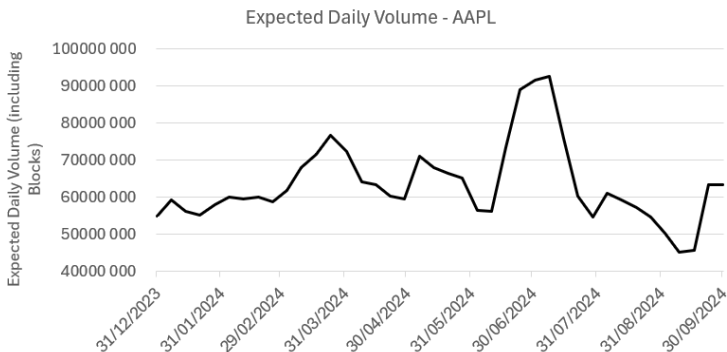
Order Type:

The type of order placed can also influence slippage. For example, market orders are designed to be filled as quickly as possible, but they do so at the best available price, which may differ from the expected price. In contrast, limit orders specify a maximum or minimum price at which the

trade will be executed, potentially avoiding slippage but with the risk of the order not being filled.

Liquidity Constraints:

Low liquidity can exacerbate slippage, as fewer buyers or sellers are available to match the order at the desired price. When liquidity is thin, even relatively small orders can move the market, resulting in slippage. This is especially true in smaller or emerging markets, where fewer participants trade large volumes of assets.



2.4 Impact on Trade Execution and Returns

The combined effects of transaction costs and slippage can have a profound impact on trade execution and overall strategy returns. Even small costs, when accumulated over thousands of trades, can significantly reduce profitability. High-frequency trading (HFT) strategies, which rely on capitalizing on small price movements, are particularly vulnerable to

these costs. A slight increase in the bid-ask spread or small slippage on multiple trades can turn a profitable algorithm into a losing one.

Moreover, the impact of these costs is often underestimated during backtesting, as many simulations do not accurately account for real-world trading conditions. For example, many traders assume they will always be able to execute trades at the exact prices they specify, ignoring the impact of slippage and market impact. This overestimation of performance can lead to misguided optimism and poor live results when the strategy is implemented in real markets.

To mitigate these issues, traders must incorporate realistic models of transaction costs and slippage into their backtesting frameworks. This can include using historical data on bid-ask spreads, factoring in potential slippage based on volatility, and simulating the market impact of large orders. By accounting for these hidden expenses in advance, traders can develop more robust and cost-effective algorithms, leading to more accurate performance estimates and improved execution strategies.

3 Impact on algorithmic trading performance

Transaction costs and slippage have varying effects on different types of trading strategies, particularly when comparing high-frequency trading (HFT) strategies with low-frequency ones. Understanding their influence is critical to developing realistic and robust trading systems.

3.1 High-Frequency Strategies

High-frequency trading strategies, which involve executing a large number of trades over short periods, are particularly sensitive to transaction costs and slippage. Given the narrow profit margins that HFT strategies aim to exploit, even small transaction costs or slippage can erode profitability.

Let P represent the profit per trade and C the transaction cost per trade. In the absence of slippage, the net profit P_{net} from a trade can be modeled as:

$$P_{\text{net}} = P - C$$

For an HFT strategy where trades occur frequently, the cumulative net profit Π_{HFT} over N trades is:

$$\Pi_{\text{HFT}} = \sum_{i=1}^N (P_i - C_i)$$

When slippage is introduced, let S represent the slippage per trade, so the net profit becomes:

$$P_{\text{net}} = P - C - S$$

The cumulative net profit over N trades with slippage included is:

$$\Pi_{\text{HFT}} = \sum_{i=1}^N (P_i - C_i - S_i)$$

As N increases, the negative impact of slippage and transaction costs compounds, which can lead to significant deviations from theoretical backtest results. For example, if P_i is small relative to C_i and S_i , the strategy may become unprofitable.

3.2 Low-Frequency Strategies

Low-frequency trading strategies, by contrast, are less affected by transaction costs and slippage because they execute fewer trades, often based on longer-term signals. The profit per trade P_{low} is generally higher in these strategies, reducing the relative impact of C and S .

For a low-frequency strategy, the cumulative profit Π_{low} over M trades, where $M \ll N$, is:

$$\Pi_{\text{low}} = \sum_{j=1}^M (P_j - C_j - S_j)$$

Since the number of trades M is smaller and P_j is generally larger, the total cost burden from transaction fees and slippage is relatively lower. However, neglecting these factors in backtesting can still result in overly optimistic performance estimates.

3.3 Impact on Backtest Accuracy

In backtesting, many strategies assume perfect trade execution at desired prices, neglecting both slippage and the variability of transaction costs. This leads to inaccurate estimates of performance. A more realistic model of returns that includes these factors can be represented as:

$$R_{\text{net}} = R_{\text{gross}} - \frac{T}{V} - \frac{S}{V}$$

where R_{gross} is the gross return of the strategy, T represents the transaction costs, S represents slippage, and V is the traded volume. Both T and S are typically functions of order size, market liquidity, and volatility.

The failure to include these real-world costs in backtesting can lead to what is known as "overfitting to theoretical performance." To mitigate this, many traders simulate realistic transaction costs and slippage using historical market data, thus aligning their backtests more closely with real-world conditions.

For example, the slippage S can be modeled as a function of the volatility σ of the asset and the size of the order relative to the average daily volume:

$$S = k \cdot \sigma \cdot \frac{O}{\text{ADV}}$$

where O is the order size, ADV is the average daily volume, and k is a constant that reflects market impact. This formula captures the fact that larger orders and higher volatility tend to increase slippage. Institutional investors could use a k of 30%, retails could keep 100% as traded volume is quite low relatively to big players (The previous is incorrect on smaller market capitalizations for which the average daily volume can drop to 10 shares a day !).

3.4 Comparative Sensitivity

The sensitivity of strategy performance to transaction costs and slippage can be evaluated by comparing their relative impact on high-frequency and low-frequency strategies. In HFT strategies, where the average profit per trade P is small, the impact of costs C and slippage S is proportionally larger:

$$\frac{C + S}{P} \approx 1$$

This means that transaction costs and slippage can effectively consume most, if not all, of the profits in an HFT strategy. In contrast, for low-frequency strategies where $P_{\text{low}} \gg C$, the ratio is much smaller:

$$\frac{C + S}{P_{\text{low}}} \ll 1$$

Thus, while both types of strategies are affected by these costs, high-frequency strategies are much more vulnerable to performance degradation due to transaction costs and slippage.

4 Mitigation Techniques

Reducing the impact of transaction costs and slippage is critical to improving the real-world performance of algorithmic trading strategies. Several techniques can be employed to mitigate these effects, including execution strategies and trade optimizations. This section explores these methods, with a focus on buffering techniques as applied to a moving average strategy.

4.1 Execution Strategies

Effective execution strategies are essential to minimizing the direct costs of trading, particularly slippage and market impact. Some common methods include:

- **Limit Orders:** Using limit orders instead of market orders helps avoid slippage by specifying the exact price at which to execute trades. However, the downside is that limit orders may not always be filled, especially in fast-moving markets.
- **Smart Order Routing (SOR):** SOR algorithms route trades across multiple venues to achieve the best possible execution price, thus reducing the bid-ask spread and improving execution quality.
- **Volume-Weighted Average Price (VWAP) and Time-Weighted Average Price (TWAP):** These algorithms execute trades incrementally over a defined period, reducing market impact by spreading the order across time and matching the average market price.

While these execution strategies are very important for controlling costs, they are not sufficient on their own. Further optimizations within the trading strategy are necessary to effectively manage the impact of transaction costs and slippage.

4.2 Buffering

Buffering is an effective technique to reduce unnecessary trades and, consequently, minimize transaction costs in trend-following strategies like moving average crossovers. In a simple moving average (SMA) crossover strategy, trades are triggered whenever a short-term moving average crosses a long-term moving average. However, frequent crossovers can result in a high number of trades, particularly during periods of low market momentum, leading to substantial transaction costs.

To address this, buffering introduces a tolerance band or "buffer" around the crossover signal, preventing trades unless the signal exceeds a certain threshold. This reduces the frequency of trades and filters out noise, thereby lowering transaction costs.

4.2.1 Mathematical Formulation of Buffering

Let $\text{SMA}_{\text{short}}(t)$ and $\text{SMA}_{\text{long}}(t)$ represent the short-term and long-term simple moving averages at time t , respectively. In a typical crossover strategy, a buy signal is generated when:

$$\text{SMA}_{\text{short}}(t) > \text{SMA}_{\text{long}}(t)$$

and a sell signal is generated when:

$$\text{SMA}_{\text{short}}(t) < \text{SMA}_{\text{long}}(t)$$

To implement buffering, we introduce a threshold δ , which defines the width of the buffer zone. Rather than executing trades on every crossover, trades are only triggered when the difference between the two moving averages exceeds δ . Specifically, a buy signal is generated when:

$$\text{SMA}_{\text{short}}(t) > \text{SMA}_{\text{long}}(t) + \delta$$

and a sell signal when:

$$\text{SMA}_{\text{short}}(t) < \text{SMA}_{\text{long}}(t) - \delta$$

This approach effectively filters out small, inconsequential crossovers that would otherwise result in costly trades.

4.2.2 Impact on Transaction Costs

By implementing a buffer, the number of trades N executed by the strategy is reduced. The total transaction cost T_{cost} for a strategy without buffering is typically given by:

$$T_{\text{cost}} = N \times C$$

where C represents the per-trade transaction cost. When buffering is applied, the number of trades N_{buffered} decreases, leading to:

$$T_{\text{cost,buffered}} = N_{\text{buffered}} \times C$$

Given that $N_{\text{buffered}} < N$, it follows that:

$$T_{\text{cost,buffered}} < T_{\text{cost}}$$

Thus, buffering can significantly reduce transaction costs by decreasing the total number of trades.

4.3 Trade Optimizations

Beyond buffering, several additional optimization techniques can help reduce the overall impact of transaction costs and slippage. These include:

- **Position Sizing Adjustments:** Implementing volatility-based position sizing can help reduce the size of trades during periods of high volatility, thereby limiting market impact and slippage.

- **Trade Clustering:** Grouping trades together to execute multiple orders simultaneously can reduce the frequency of trading and minimize transaction costs.
- **Rebalancing Thresholds:** Similar to buffering, rebalancing thresholds can be set to avoid constant portfolio rebalancing, reducing transaction costs associated with frequent adjustments.

4.4 Strategy Example

4.5 Moving Average Strategy with Buffering

In this subsection, we will implement a simple moving average-based strategy on APPLE stock with a buffering mechanism to reduce unnecessary trades and mitigate the impact of transaction costs and slippage. The strategy relies on two moving averages: a 20-day short-term moving average MA_{20} and a 50-day long-term moving average MA_{50} .

The basic trading rule is as follows:

- **Buy Signal:** We will enter a long position (buy the underlying asset) when the 20-day moving average exceeds the 50-day moving average:

$$MA_{20} > MA_{50}$$

- **Exit to Cash:** If the 20-day moving average is less than or equal to the 50-day moving average, we will exit the position and remain in cash.

This basic rule can lead to frequent trades, especially during periods of low market volatility, where small price fluctuations can cause multiple crossovers between the two moving averages. To mitigate this, we introduce a buffering mechanism that creates a tolerance zone around the crossover point, preventing trades unless a significant price movement occurs.

The buffer is defined as a multiple of the 30-day price standard deviation $\text{Price}_{\text{std}30}$, which acts as a measure of recent market volatility. Specifically, the buffer size is set as:

$$\text{Buffer} = 2 \times \text{Price}_{\text{std}30}$$

The introduction of the buffer modifies the trading rules as follows:

- **Buy Signal with Buffer:** We will only enter a long position when the 20-day moving average exceeds the 50-day moving average by more than the buffer value:

$$MA_{20} > MA_{50} + \text{Buffer}$$

This ensures that we only enter the market when there is a strong enough signal, reducing the number of trades triggered by noise.

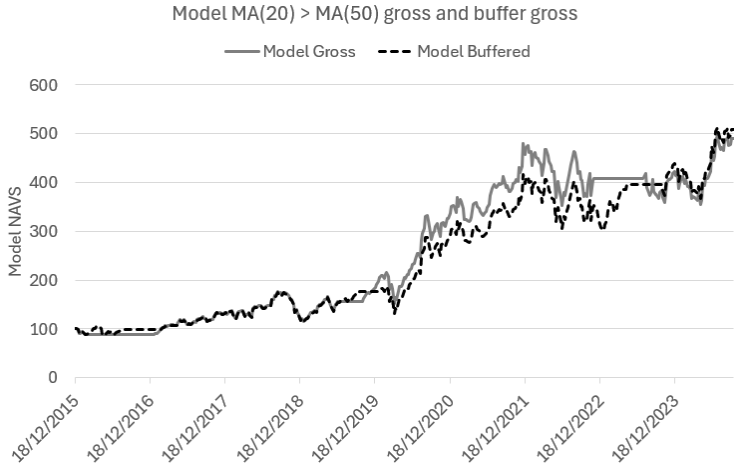
- **Exit to Cash with Buffer:** Similarly, we will only exit the position and return to cash when the 20-day moving average falls below the 50-day moving average by more than the buffer value:

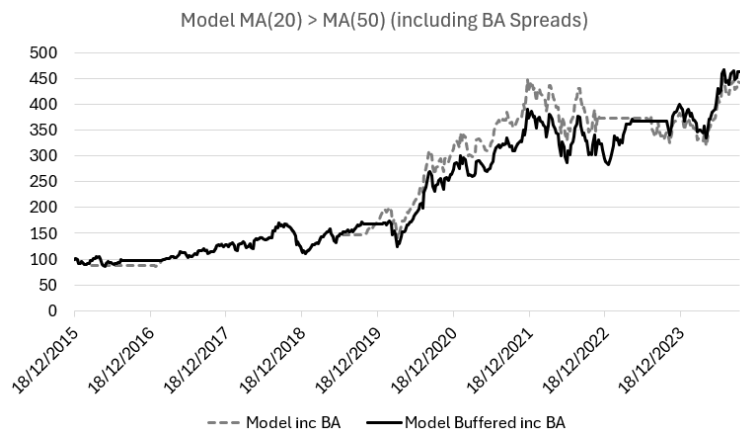
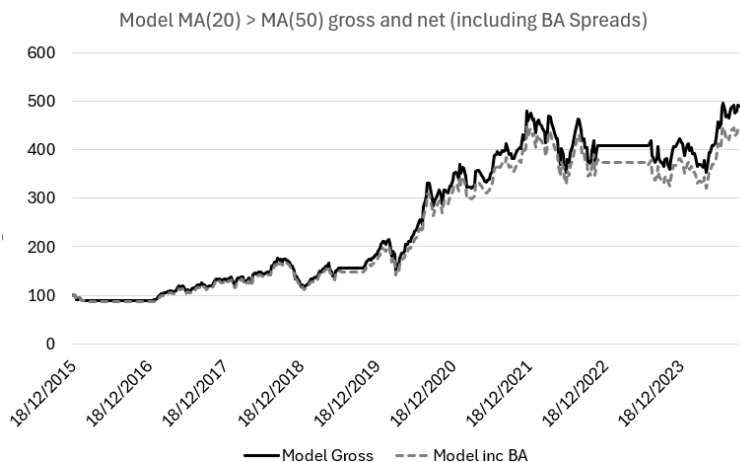
$$MA_{20} < MA_{50} - \text{Buffer}$$

This prevents premature exits caused by minor fluctuations.

The choice of the buffer size being a multiple of the price's 30-day standard deviation is based on the notion that higher volatility environments are likely to produce larger short-term price swings, which could result in more frequent, but less meaningful, crossovers. By setting the buffer proportional to volatility, we adjust the strategy dynamically based on the market's recent behavior. In low volatility conditions, the buffer will be smaller, allowing the strategy to react more quickly to genuine trends.

Conversely, in high volatility conditions, the buffer will be larger, helping to filter out short-term noise and avoid overtrading.





By introducing the buffer, the frequency of trades is reduced. As described in the previous section, this leads to a reduction in transaction costs, since fewer trades are executed. The total number of trades N_{buffered} will be smaller than without the buffer, and hence the cumulative transaction costs will also decrease:

$$T_{\text{cost,buffered}} = N_{\text{buffered}} \times C$$

where C is the per-trade transaction cost. As the number of trades N_{buffered} decreases due to the buffer, the total costs associated with the strategy will be lower, improving the net profitability of the strategy.

Table 1: Backtest Results Comparison (Model Gross vs. Model including BA Costs)

Metric	Model Gross	Model including BA Costs
Time in Market	80.0%	80.0%
Cumulative Return	390.15%	343.12%
CAGR	13.34%	12.44%
Sharpe Ratio	1.84	1.74
Smart Sharpe	1.81	1.72
Sortino Ratio	2.84	2.68
Omega Ratio	1.42	1.39
Max Drawdown	-34.64%	-34.64%
Volatility (ann.)	56.01%	56.05%
R^2	0.84	0.84
Information Ratio	-0.09	-0.10
Calmar Ratio	0.39	0.36
Skewness	-0.06	-0.05
Kurtosis	3.04	3.02
Daily Value-at-Risk	-5.39%	-5.42%
cVaR	-5.39%	-5.42%
Payoff Ratio	0.97	0.98
Profit Factor	1.42	1.39
Average Drawdown	-8.28%	-8.56%

Table 2: Backtest Results Comparison (Model Buffered vs. Model Buffered including BA Costs)

Metric	Model Buffered	Buffered with BA Costs
Time in Market	86.0%	87.0%
Cumulative Return	408.09%	363.1%
CAGR	13.66%	12.83%
Sharpe Ratio	1.84	1.75
Smart Sharpe	1.82	1.73
Sortino Ratio	2.83	2.69
Omega Ratio	1.40	1.38
Max Drawdown	-34.64%	-34.64%
Volatility (ann.)	57.62%	57.69%
R^2	0.90	0.89
Information Ratio	-0.10	-0.11
Calmar Ratio	0.39	0.37
Skewness	-0.10	-0.09
Kurtosis	2.65	2.62
Daily Value-at-Risk	-5.55%	-5.58%
cVaR	-5.55%	-5.58%
Payoff Ratio	1.05	1.05
Profit Factor	1.40	1.38
Average Drawdown	-8.70%	-9.00%

5 Comments on the Backtest Results

Time in Market: The buffered models show a slightly higher time in market (86-87%) compared to the unbuffered models (80%). This suggests that buffering extends the holding periods slightly, which may be due to fewer small exit signals, reducing noise trading.

Cumulative Return and CAGR: The Model Buffered has the highest cumulative return (408.09%) and CAGR (13.66%), indicating that buffering adds value even after accounting for costs, with only a slight drop in performance when bid-ask (BA) costs are included. The impact of BA costs is visible across both buffered and unbuffered models, but buffered strategies mitigate the cost impacts better.

Risk-Adjusted Ratios (Sharpe, Sortino, Omega): Sharpe, Sortino, and Omega ratios remain largely stable across models, indicating that the overall risk-adjusted returns of the strategy are consistent. While the buffered models marginally improve the Sortino and Omega ratios, the changes are not drastic.

Drawdown and Risk Measures: The max drawdown remains constant across all models at -34.64%, indicating that the worst drawdown period is unaffected by the introduction of buffering or BA costs. However, the average drawdown is slightly higher in the buffered models, showing a minor trade-off between higher returns and increased drawdown.

Volatility and Correlation (R^2): Volatility increases slightly in the buffered models (57.62-57.69%) as the strategy holds positions longer. The R^2 value increases for the buffered models, indicating a better fit between the model and the benchmark after buffering.

Skewness and Kurtosis: Both skewness and kurtosis decrease slightly in the buffered models. A reduction in kurtosis shows that returns are less prone to extreme outliers after buffering, which contributes to smoother return distributions.

Payoff Ratio and Profit Factor: The buffered models show a notable improvement in the payoff ratio (1.05 vs. 0.97-0.98), meaning that buffered trades tend to generate larger profits relative to losses. This is confirmed by the stable profit factor across models, suggesting consistent profitability.

Conclusion: Buffering generally improves performance by increasing cumulative returns, improving the payoff ratio, and slightly enhancing risk-adjusted metrics like the Sortino ratio. However, this comes at the cost of slightly higher average drawdown and volatility. BA costs affect all models but are mitigated more effectively in buffered strategies. The results indicate that buffering can be a valuable technique for reducing noise trades, improving overall profitability, and optimizing the balance between risk and return.

6 Conclusion

In this document, we have explored the impact of transaction costs and slippage on algorithmic trading performance, focusing on the significance of these factors in enhancing trading strategies. Through comprehensive backtesting and quantitative analysis, we demonstrated that neglecting transaction costs can lead to overestimating the efficacy of trading models, thereby misguiding decision-making processes.

Our findings reveal that incorporating transaction costs into the evaluation of trading strategies not only provides a more accurate representation of performance but also emphasizes the importance of optimizing execution strategies to minimize slippage. The developed models showcased how robust backtesting methodologies can significantly improve the risk-return profiles of algorithmic trading systems.

Furthermore, we discussed the implications of our results for practitioners in the field of algorithmic trading. By understanding the critical role of transaction costs and slippage, traders can develop more effective strategies that align with real-world trading conditions, ultimately leading to enhanced profitability.

Future research could explore the integration of machine learning techniques to further refine trading strategies, addressing transaction costs in real-time trading environments. This work serves as a foundation for developing more resilient algorithmic trading frameworks that can withstand market fluctuations while maintaining optimal performance.

In conclusion, the insights gained from this research highlight the necessity for traders and researchers alike to incorporate transaction costs and slippage into their analyses to achieve realistic and sustainable trading success.